



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Kim Dinh Son (www.sonkd.im | sonkdbk@gmail.com)
Jan 29, 2026



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Summary of methodologies
 - Data integration and transformation: Collect data through APIs and web scraping, followed by data wrangling and preprocessing.
 - Descriptive and diagnostic analysis (EDA): Use data visualization to identify feature relationships that strongly influence the target variable.
 - Machine learning modeling: Apply machine learning techniques to build models that predict the key target outcomes.
- Summary of all results:
 - Launch sites are typically located near railways, roads, highways, and coastlines to support logistics.
 - Payloads in the 2,500–5,000 kg range account for the majority of successful launches.
 - The Decision Tree model achieves the highest accuracy among all classifiers when predicting first-stage landing success using the machine learning pipeline.

Introduction

- Project background and context:
Space X advertises Falcon 9 rocket launches on its website with a cost of 62 million dollars; other providers cost upward of 165 million dollars each, much of the savings is because Space X can reuse the first stage. Therefore if we can determine if the first stage will land, we can determine the cost of a launch. This information can be used if an alternate company wants to bid against space X for a rocket launch. As a data science, I will create a machine learning pipeline to predict if the first stage will land given the data from the preceding labs.
- Problems that I want to find answers:
 - What are the key factors that contribute to successful rocket landings?
 - How do different features interact to influence launch success rates and first-stage landing success rates?

Section 1

Methodology

Methodology

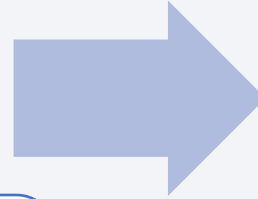
Executive Summary

- Data collection methodology:
 - Request to the SpaceX API. Clean the requested data
- Perform data wrangling
 - I applied the web scraping method to collect Falcon 9 historical launch records from a Wikipedia page: https://en.wikipedia.org/wiki/List_of_Falcon_9_and_Falcon_Heavy_launches. Extract a Falcon 9 launch records HTML table from Wikipedia. Parse the table and convert it into a Pandas data frame
 - One-hot encoding was applied to categorical features
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
 - Use machine learning to determine the best classification model for predicting successful Falcon 9 first-stage landings.

Data Collection

Get the data

- Get request to the SpaceX API
- Decode the response content as Json and turn it into a pandas dataframe by using `.json_normalize()`
- Clean the data, check and fill the missing or null values



Scraping

- Extract the launch records as HTML table
- Parse the table and convert it to a pandas dataframe

Data Collection – SpaceX API

- Link to the notebook (Github):
<https://github.com/sonkd/python-project-for-data-science/blob/main/Project/jupyter-labs-spacex-data-collection-api.ipynb>

Get request to the SpaceX API

```
[7] spacex_url="https://api.spacexdata.com/v4/launches/past"

[11] response=requests.get(static_json_url)
```



Decode the response content as Json and turn it into a pandas dataframe by using .json_normalize()

```
[18] data = response.json()
      data = pd.json_normalize(data)
```



Clean the data, check and fill the missing or null values

```
[66] data_falcon9.isnull().sum()

[67] # Calculate the mean value of PayloadMass column
      payload_mean = data_falcon9['PayloadMass'].mean()

      # Replace the np.nan values with its mean value
      data_falcon9['PayloadMass'] = data_falcon9['PayloadMass'].replace(np.nan, payload_mean)

[64] data_falcon9 = pd.DataFrame(launch_dict_data[launch_dict_data['BoosterVersion']!='Falcon 1'])

[65] data_falcon9.loc[:, 'FlightNumber'] = list(range(1, data_falcon9.shape[0]+1))
      data_falcon9
```


Data Collection - Scraping

- Link to the notebook (Github):
<https://github.com/sonkd/python-project-for-data-science/blob/main/Project/jupyter-labs-webscraping.ipynb>

Extract the launch records as HTML table

```
static_url = "https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Falcon_Heavy_launches&oldid=1027686922"

headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
    "AppleWebKit/537.36 (KHTML, like Gecko) "
    "Chrome/91.0.4472.124 Safari/537.36"
}

# use requests.get() method with the provided static_url and headers
# assign the response to a object
response = requests.get(static_url, headers = headers)

# Use the find_all function in the BeautifulSoup object, with element type 'table'
# Assign the result to a list called 'html_tables'
html_tables = soup.find_all('table')

# Let's print the third table and check its content
first_launch_table = html_tables[2]
```



Parse the table and convert it to a pandas dataframe

```
column_names = []

# Apply find_all() function with 'th' element on first_launch_table
# Iterate each th element and apply the provided extract_column_from_header() to get a column name
# Append the Non-empty column name ('if name is not None and len(name) > 0') into a list called column_names

cols = first_launch_table.find_all('th')

for th in cols:
    name = extract_column_from_header(th)

    if name is not None and len(name) > 0:
        column_names.append(name)

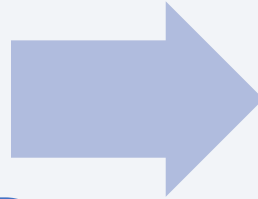
launch_dict= dict.fromkeys(column_names)

df= pd.DataFrame({ key:pd.Series(value) for key, value in launch_dict.items() })
```

Data Wrangling

Exploratory Data Analysis

- Identify and calculate the percentage of the missing values in each attribute (identify which columns are numerical and categorical)
- Calculate the number of launches on each site
- Calculate the number and occurrence of each orbit
- Calculate the number and occurrence of mission outcome of the orbits



Determine Training Labels

- Create a landing outcome label from Outcome column.
- Set a column named "Class" with 0 as the "fail" and 1 as "success" result.

- Link to the notebook (Github): <https://github.com/sonkd/python-project-for-data-science/blob/main/Project/labs-jupyter-spacex-Data%20wrangling.ipynb>

EDA with Data Visualization

- My aim is to prepare the data through feature engineering. Therefore, I need to apply an EDA step to understand which features have a strong relationship with the target (the outcome column is "Class").
- My process:
 - Use Scatter plot to visualize the relationship between the pairs: (Flight Number, Payload), (Flight Number, Launch Site), (Launch Site, Payload)
 - Use Bar chart to visualize the success rate of each Orbit type
 - Use Scatter plot to visualize the relationship between Flight Number and Orbit type, Payload and Orbit type
 - Use Line chart to visualize the launch success yearly trend
- Link to the notebook (Github): <https://github.com/sonkd/python-project-for-data-science/blob/main/Project/edadataviz.ipynb>

EDA with SQL

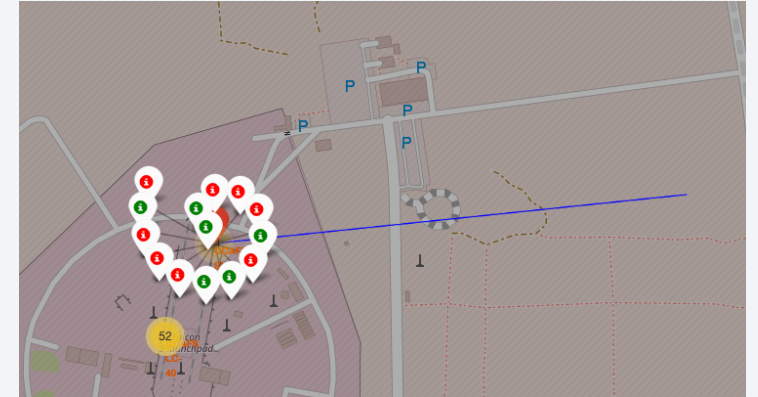
- Load the dataset into the corresponding table in a database
 1. Install the libraries: sqlalchemy, ipython-sql, prettytable. Import csv, sqlite3 and prettytable.
 2. Load data (.csv file) from database into a pandas dataframe (using to_sql function)
 3. Create the table SPACE TABLE for working with data with SQL
- Explore data

Check the names of the unique launch sites, display the total payload mass carried by boosters launched by NASA, list the date when the first successful landing outcome in ground pad, see the total number of successful and failure mission outcome.

Rank the count of landing outcomes
- Link to the notebook (Github): https://github.com/sonkd/python-project-for-data-science/blob/main/Project/jupyter-labs-eda-sql-coursera_sqlite.ipynb

Build an Interactive Map with Folium

- I marked all launch sites on a map (`site_map = folium.Map()`). I created the circle for each launch site based on its coordinates.
- Then I marked the success/failed launches for each site on the map. I added a new "`marker_color`" column (red is fail and green is success) from data table (`spacex_df`) then add an icon (`folium.Icon()`) with `icon_color = record['marker_color']`
- After that, I calculated the distances between a launch site to its proximitites. I drawn a line (`PolyLine`) from that launch site to the selected coastline point.
- My purposes: understand the relationship between the external objects from real world (such as railways, highways and coastlines) and the launch site.
- Link to the notebook (Github):
https://github.com/sonkd/python-project-for-data-science/blob/main/Project/lab_jupyter_launch_site_location.ipynb



Build a Dashboard with Plotly Dash

- I built an interactive dashboard that includes a pie chart showing the launch-site success rate and a scatter plot showing the relationship between Outcome and Payload Mass (Kg) across different booster versions.
- Those plots show all the results from my understanding of SpaceX launch data. Include the key components represented for the target features (launch sites with their outcomes) and the relationship with the key features.
- Link to the notebook (Github): <https://github.com/sonkd/python-project-for-data-science/blob/main/Project/spacex-dash-app.py>

Predictive Analysis (Classification)

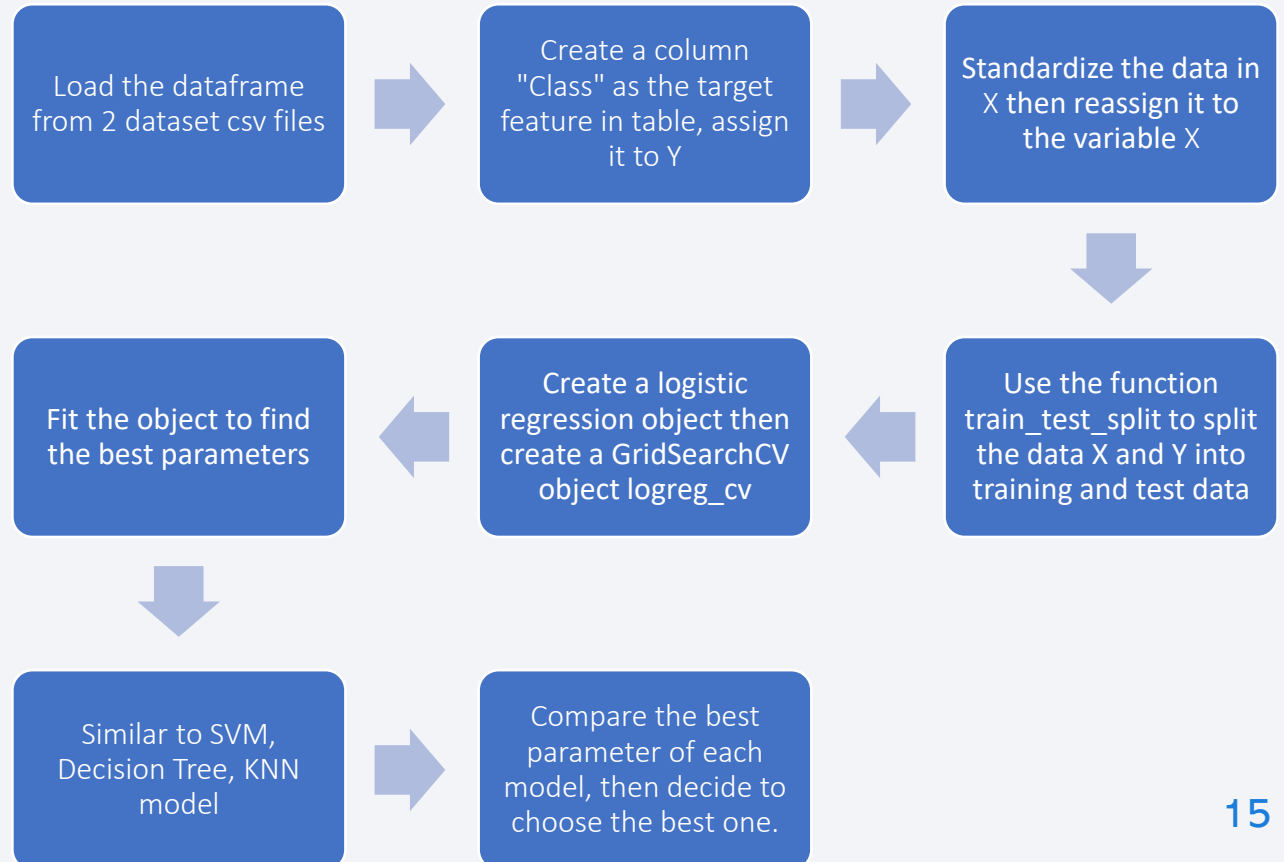
My approach:

- create a column for the class
- Standardize the data
- Split into training data and test data
- Find best Hyperparameter for SVM, Classification Trees and Logistic Regression
- Find the method performs best using test data

Link to the notebook (Github):

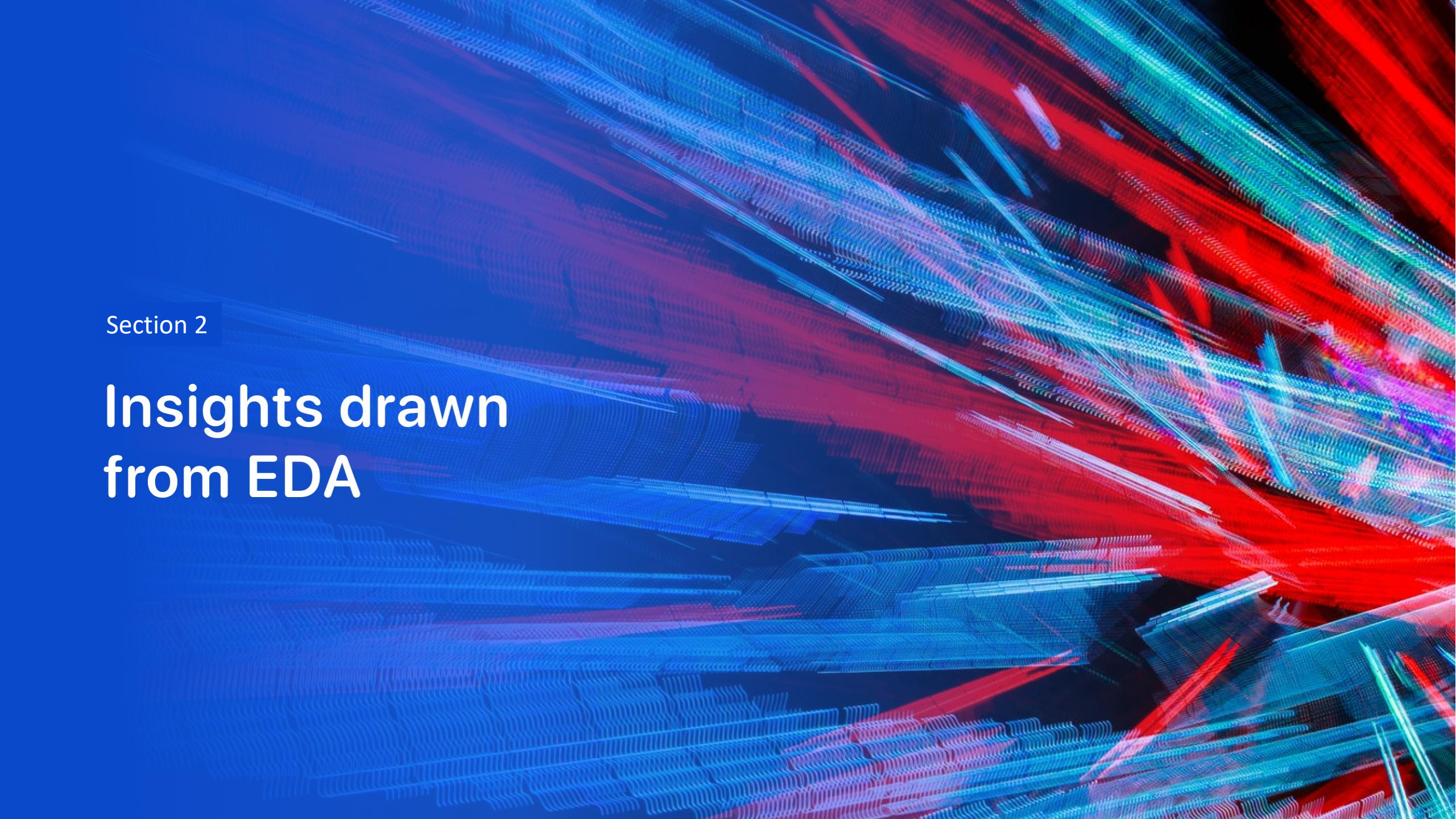
https://github.com/sonkd/python-project-for-data-science/blob/main/Project/SpaceX_Machine%20Learning%20Prediction_Part_5.ipynb

My process:



Results

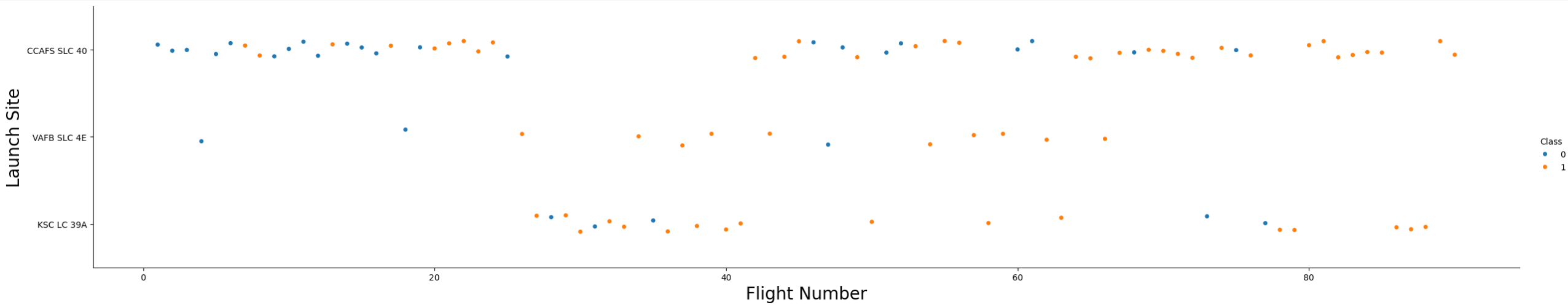
- Exploratory data analysis results
- Interactive analytics demo in screenshots
- Predictive analysis results

The background of the slide is an abstract composition. It features a dark blue base color. Overlaid on this are numerous diagonal streaks in shades of blue and red, creating a sense of motion or data flow. A faint, light blue grid pattern is also visible, particularly in the lower-left quadrant. The overall effect is high-tech and digital.

Section 2

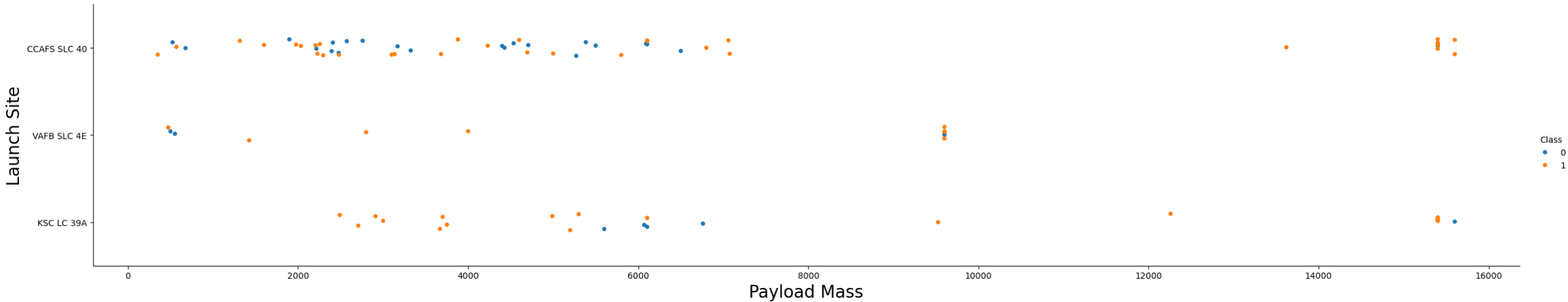
Insights drawn from EDA

Flight Number vs. Launch Site



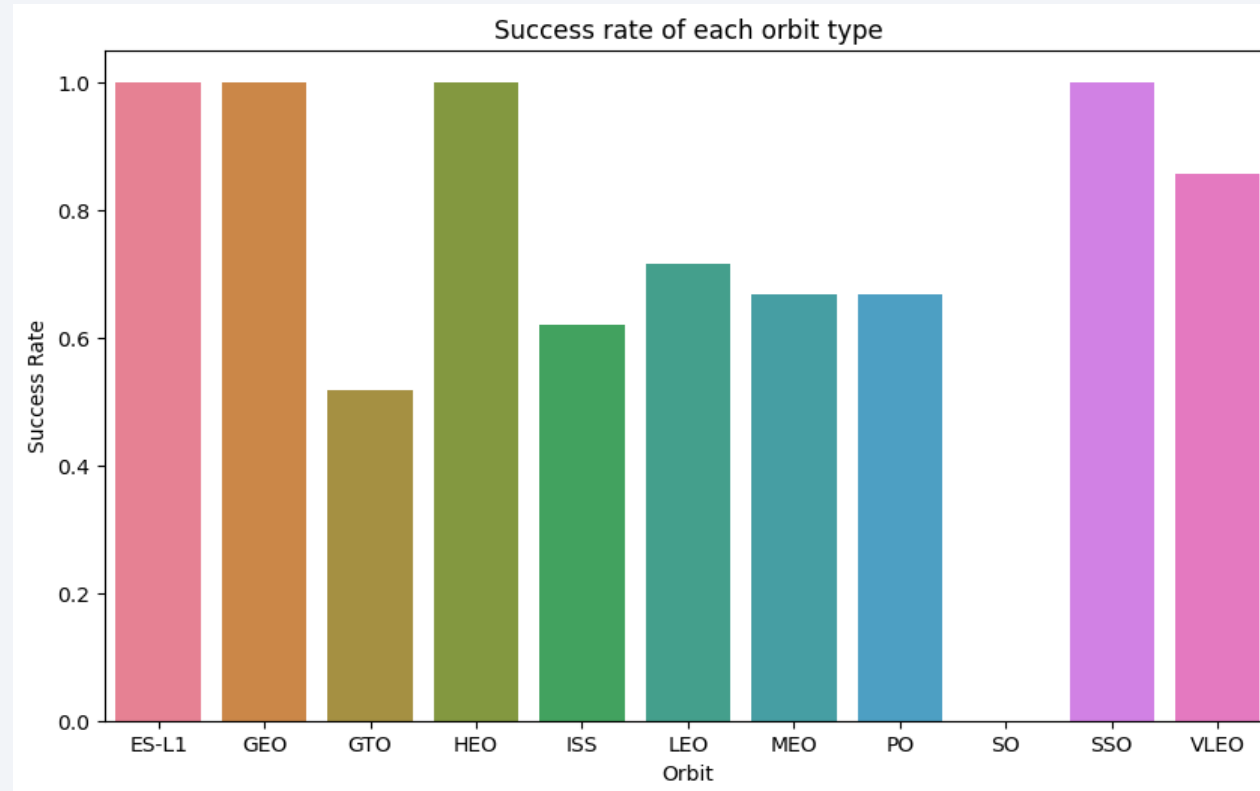
The flight number increase, the first stage is more likely to land successfully

Payload vs. Launch Site



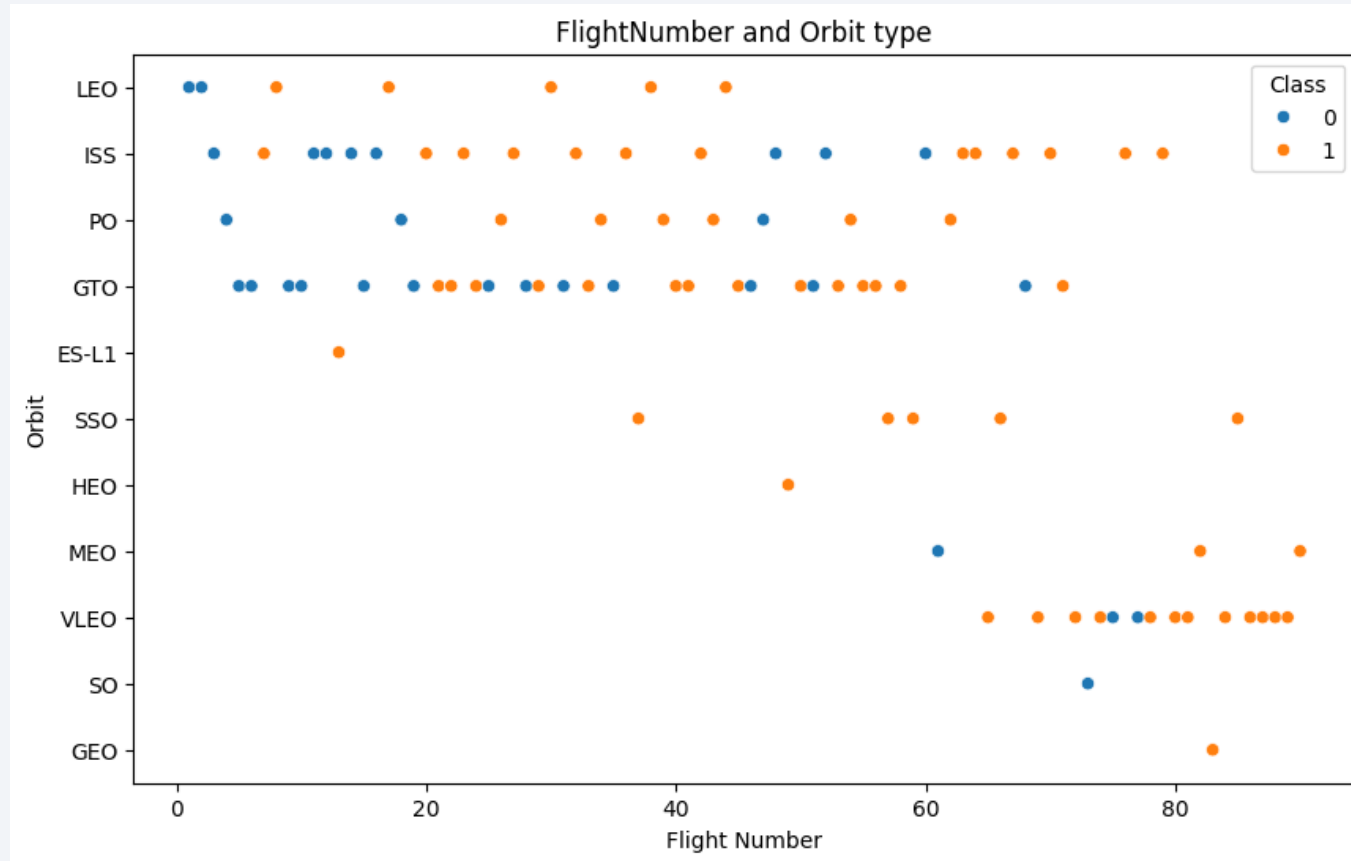
The VAFB-SLC launch site there are no rockets launched for heavy payload mass (≥ 10000)

Success Rate vs. Orbit Type



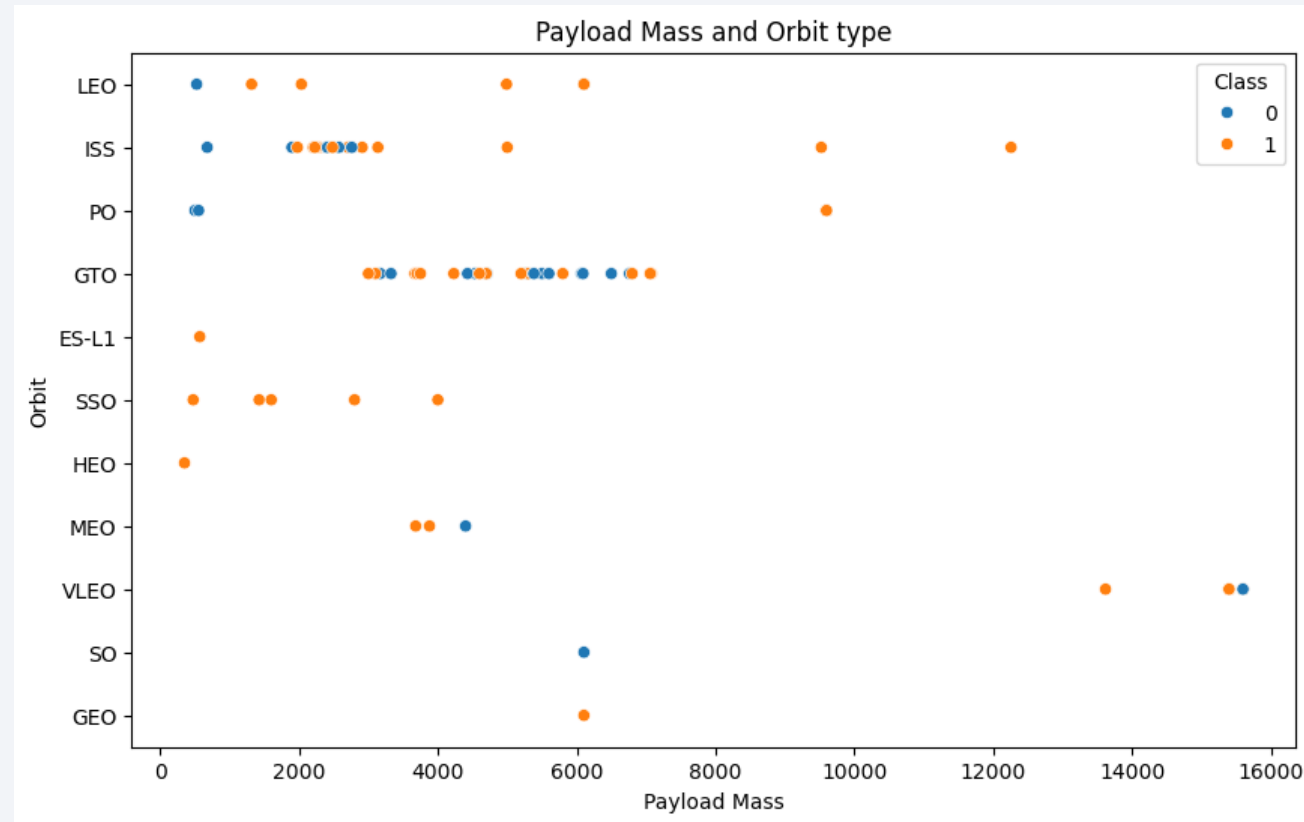
VLEO orbit with the total 14 launches have the highest success rate

Flight Number vs. Orbit Type



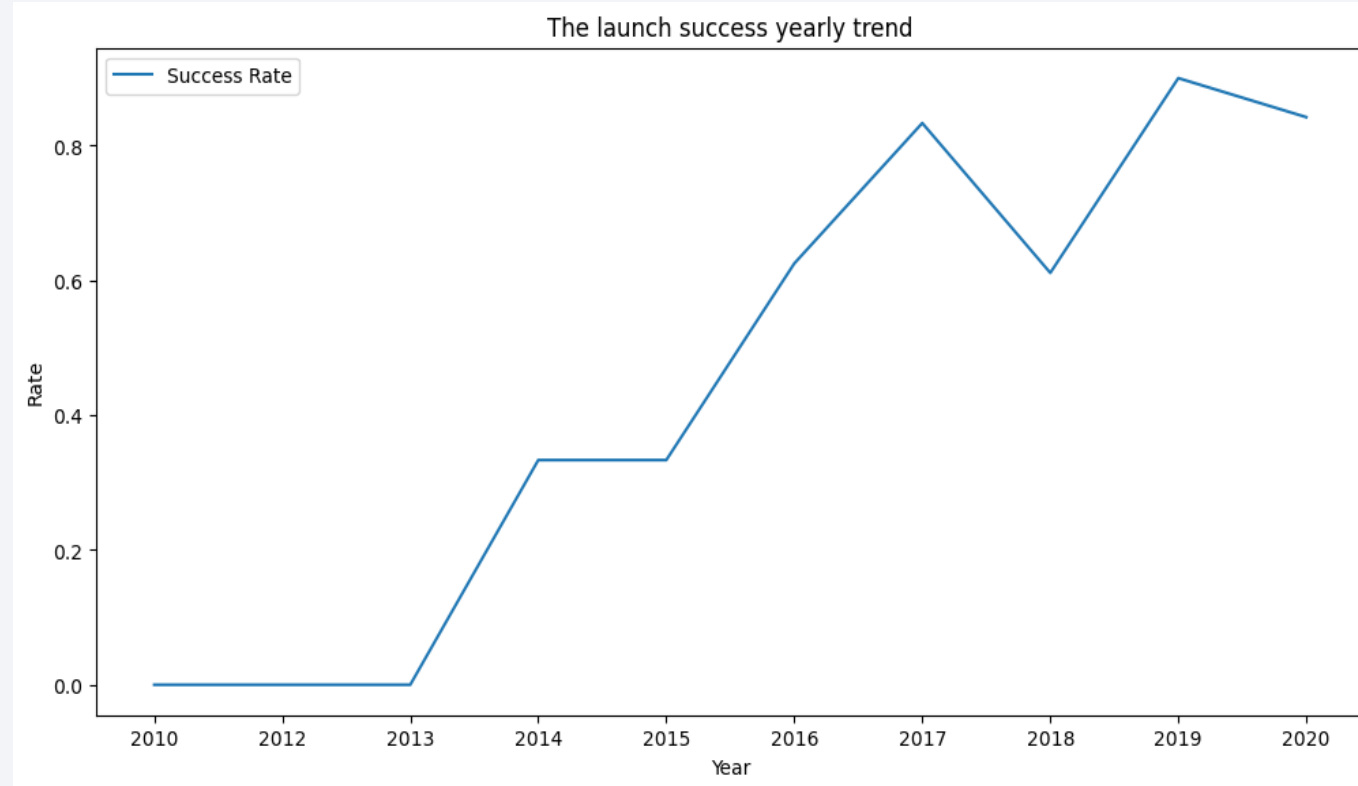
In the LEO orbit, success seems to be related to the number of flights. Conversely, in the GTO orbit, there appears to be no relationship between flight number and success.

Payload vs. Orbit Type



With heavy payloads the successful landing or positive landing rate are more for Polar, LEO and ISS. However, for GTO, it's difficult to distinguish between successful and unsuccessful landings as both outcomes are present.

Launch Success Yearly Trend



Success rate since 2013 kept increase till 2020

All Launch Site Names

- 4 launch sites are: CCAFS LC-40, VAFB SLC-4E, KSC LC-39A, CCAFS SLC-40
- Use “DISTINCT” statement to filter the final result to get the unique launch sites in the space mission.

```
%sql select distinct Launch_Site from SPACEXTABLE
```

```
* sqlite:///my_data1.db  
Done.
```

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

Launch Site Names Begin with 'CCA'

- Use the WHERE clause to query records where the value in the column Launch_Site begins with "CCA".

```
%sql select * from SPACEXTABLE where Launch_Site LIKE 'CCA%' limit 5
```

* sqlite:///my_data1.db
Done.

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Total Payload Mass

- Total is 45596Kg
- Filter the results using the condition Customer = “NASA (CRS)”, then compute the sum of PAYLOAD_MASS_KG.

```
%sql select sum(PAYLOAD_MASS_KG_) as TOTAL_PAYLOAD_MASS from SPACE_TABLE where Customer = 'NASA (CRS)'
```

```
* sqlite:///my_data1.db
```

```
Done.
```

TOTAL_PAYLOAD_MASS

45596

Average Payload Mass by F9 v1.1

- The average is 2928.4 KG

```
%sql select avg(PAYLOAD_MASS_KG_) from SPACEXTABLE where Booster_Version LIKE 'F9 v1.1'
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
avg(PAYLOAD_MASS_KG_)
```

```
2928.4
```

First Successful Ground Landing Date

- First successful landing outcome on ground pad: **2015-12-22**
- Use **min(date)** function to get the result quickly

```
%sql select min(date) from SPACEXTABLE where Landing_Outcome = 'Success (ground pad)'
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
min(date)
```

```
2015-12-22
```

Successful Drone Ship Landing with Payload between 4000 and 6000

- List the names of boosters which have successfully landed on drone ship and had payload mass greater than 4000 but less than 6000
- Use the command **between X and Y** when filter the results in a range.

```
%%sql
select
    Booster_Version,
    PAYLOAD_MASS_KG_,
    Landing_Outcome
from SPACEXTABLE
where
    Landing_Outcome = 'Success (drone ship)' and (PAYLOAD_MASS_KG_ between 4000 and 6000)
```

* sqlite:///my_data1.db

Done.

Booster_Version	PAYLOAD_MASS_KG_	Landing_Outcome
F9 FT B1022	4696	Success (drone ship)
F9 FT B1026	4600	Success (drone ship)
F9 FT B1021.2	5300	Success (drone ship)
F9 FT B1031.2	5200	Success (drone ship)

Total Number of Successful and Failure Mission Outcomes

- Failure: 1, Success: 99
- Use **count** function to get the total number after group the Mission_Outcome by using **group by** function

```
%sql select Mission_Outcome, count(Mission_Outcome) as Total from SPACEXTABLE group by Mission_Outcome
```

```
* sqlite:///my_data1.db  
Done.
```

Mission_Outcome	Total
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

Boosters Carried Maximum Payload

- Use **distinct** function to listing the different values in the column.
- Use **max** function to get the maximum value in the column.

```
%%sql
select
    distinct(Booster_Version),
    (select max(PAYLOAD_MASS__KG_) from SPACEXTABLE) as Maximum_Payload_Mass
from SPACEXTABLE
where
    PAYLOAD_MASS__KG_ = (select max(PAYLOAD_MASS__KG_) from SPACEXTABLE)
```

* sqlite:///my_data1.db

Done.

Booster_Version	Maximum_Payload_Mass
F9 B5 B1048.4	15600
F9 B5 B1049.4	15600
F9 B5 B1051.3	15600
F9 B5 B1056.4	15600
F9 B5 B1048.5	15600
F9 B5 B1051.4	15600
F9 B5 B1049.5	15600
F9 B5 B1060.2	15600
F9 B5 B1058.3	15600
F9 B5 B1051.6	15600
F9 B5 B1060.3	15600
F9 B5 B1049.7	15600

2015 Launch Records

- List the failed landing_outcomes in drone ship, their booster versions, and launch site names for in year 2015
- Use **substr(Date, x, y)** to get the name of month or day when filter in WHERE statement

```
%%sql
SELECT
    Date,
    /*CASE substr(Date, 6, 2)
        WHEN '01' THEN 'January'
        WHEN '02' THEN 'February'
        WHEN '03' THEN 'March'
        WHEN '04' THEN 'April'
        WHEN '05' THEN 'May'
        WHEN '06' THEN 'June'
        WHEN '07' THEN 'July'
        WHEN '08' THEN 'August'
        WHEN '09' THEN 'September'
        WHEN '10' THEN 'October'
        WHEN '11' THEN 'November'
        WHEN '12' THEN 'December'
    END as month, */
    Booster_Version,
    Launch_Site,
    Landing_Outcome
FROM SPACEXTABLE
WHERE substr(Date, 0, 5) = '2015' -- Lấy năm 2015
    AND Landing_Outcome LIKE '%Failure (drone ship)%' -- Thất bại hạ cánh
ORDER BY Date;
```

* sqlite:///my_data1.db

Done.

Date	Booster_Version	Launch_Site	Landing_Outcome
2015-01-10	F9 v1.1 B1012	CCAFS LC-40	Failure (drone ship)
2015-04-14	F9 v1.1 B1015	CCAFS LC-40	Failure (drone ship)

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- Use **count(*)** to calculate total times of fail/success/so on. Then use **order by** to rank those results.

```
%%sql
select Landing_Outcome, count(*) as rank
from SPACEXTABLE
    where Date between '2010-06-04' and '2017-03-20'
group by Landing_Outcome
order by rank DESC
```

```
* sqlite:///my_data1.db
Done.
```

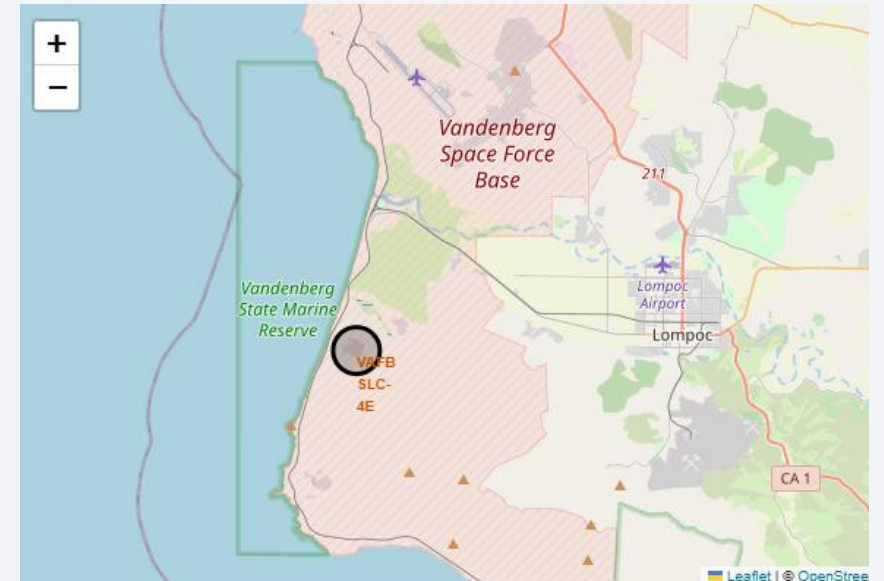
Landing_Outcome	rank
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The image is a deep blue, with the horizon line visible. The city lights are concentrated in the lower right quadrant, showing a dense network of urban areas. The text "Section 3" is overlaid on the left side of the image.

Section 3

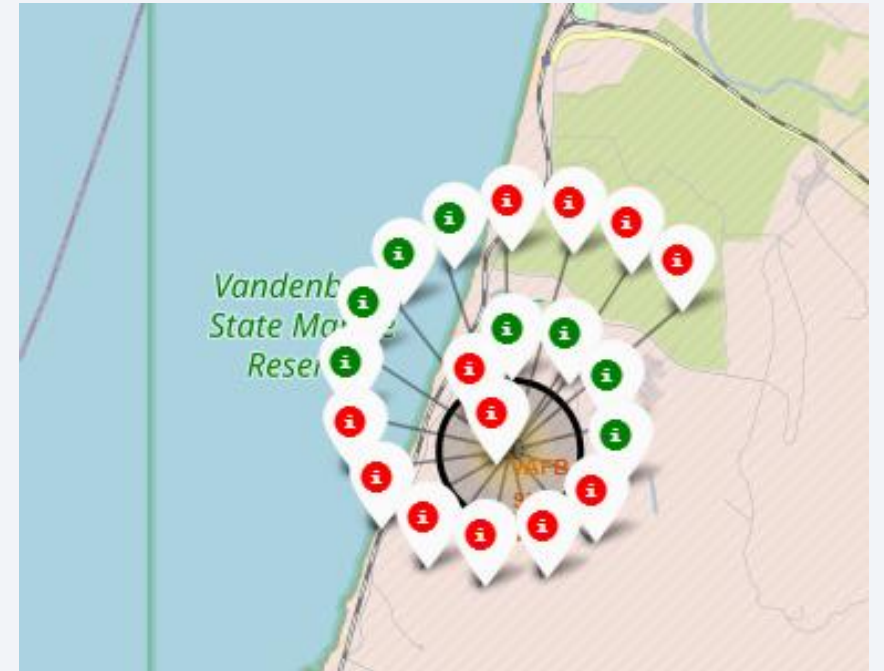
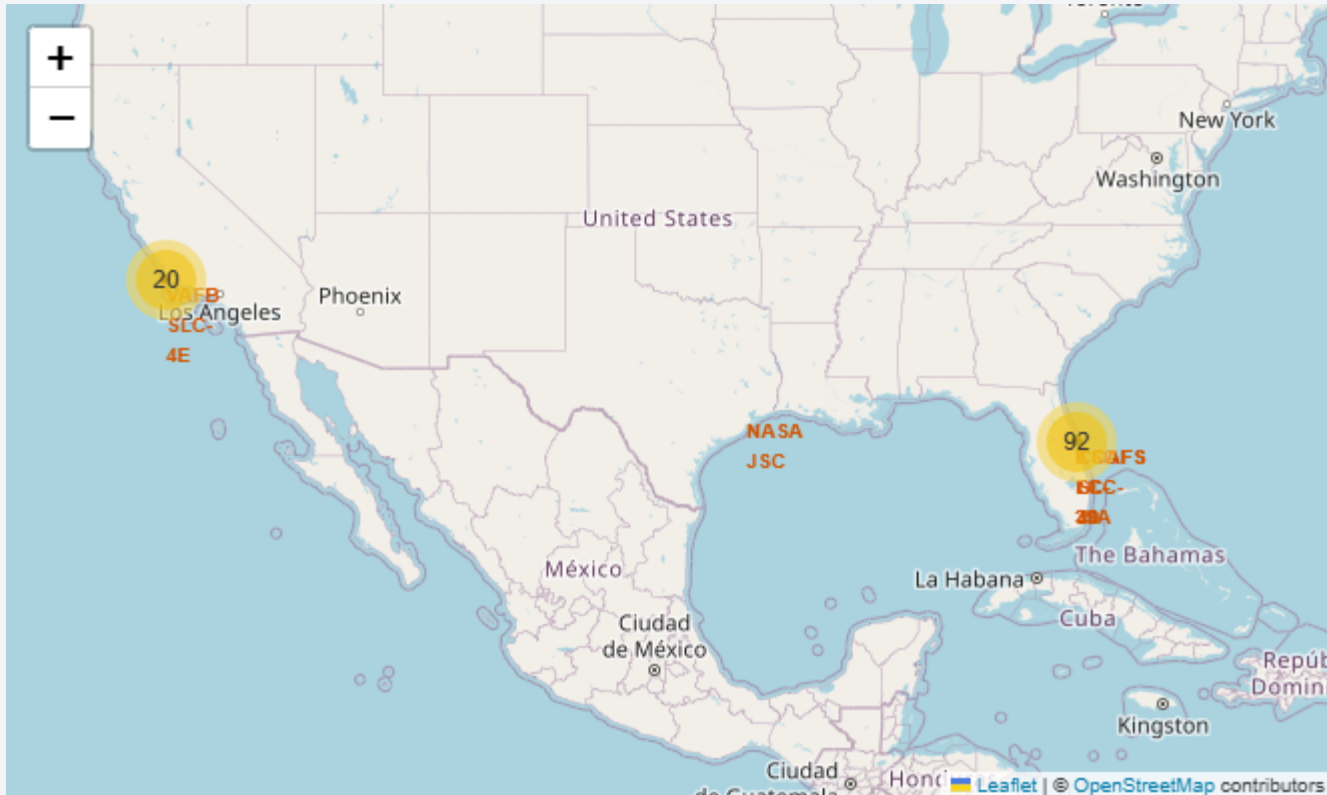
Launch Sites Proximities Analysis

All Launch Sites Map



- All launch sites are very close proximity to the coastline and into restricted areas.

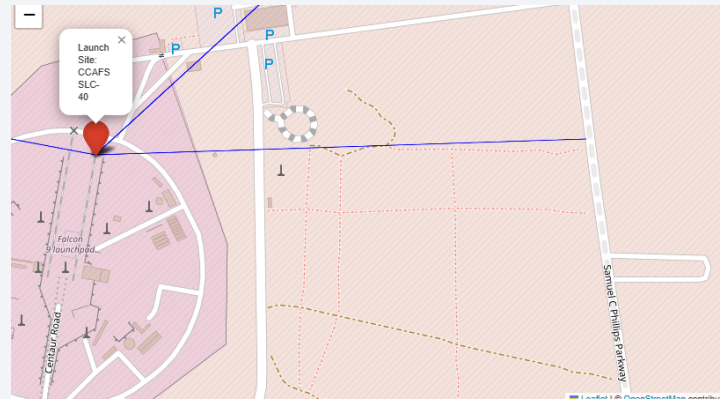
Success/Failure Launches For Each Site



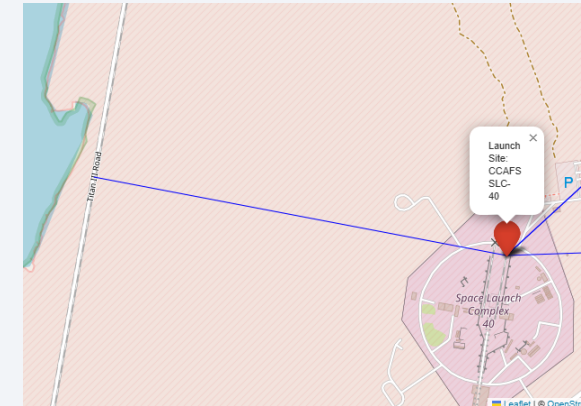
- The first map shows clusters for every launch site.
- The second shows a green marker if a launch was successful, and a red marker if a launch was failed.

Launch Site CCAFS SLC-40 And Its Proximities

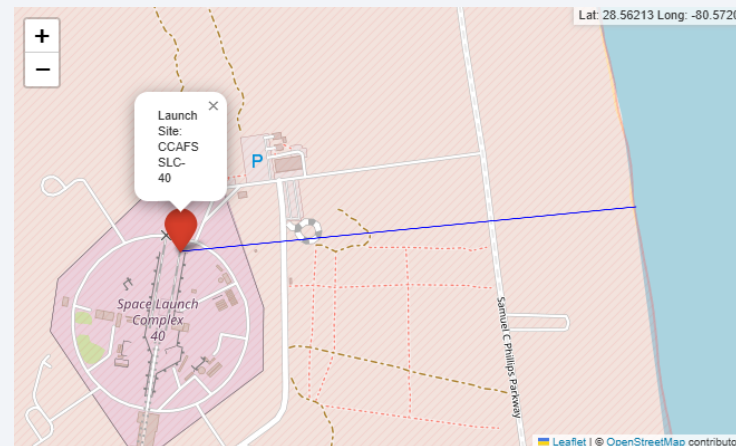
- The launch sites are located near railways, roads, highways, and the coastline.
- This placement allows for easy transportation and supply while keeping the sites far from densely populated urban areas.



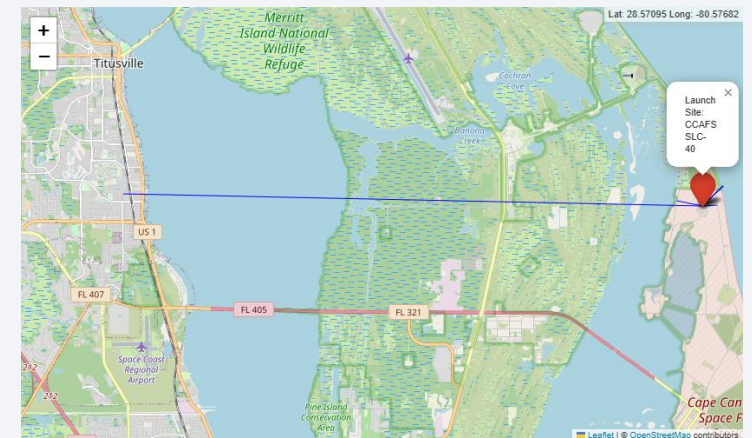
Highways



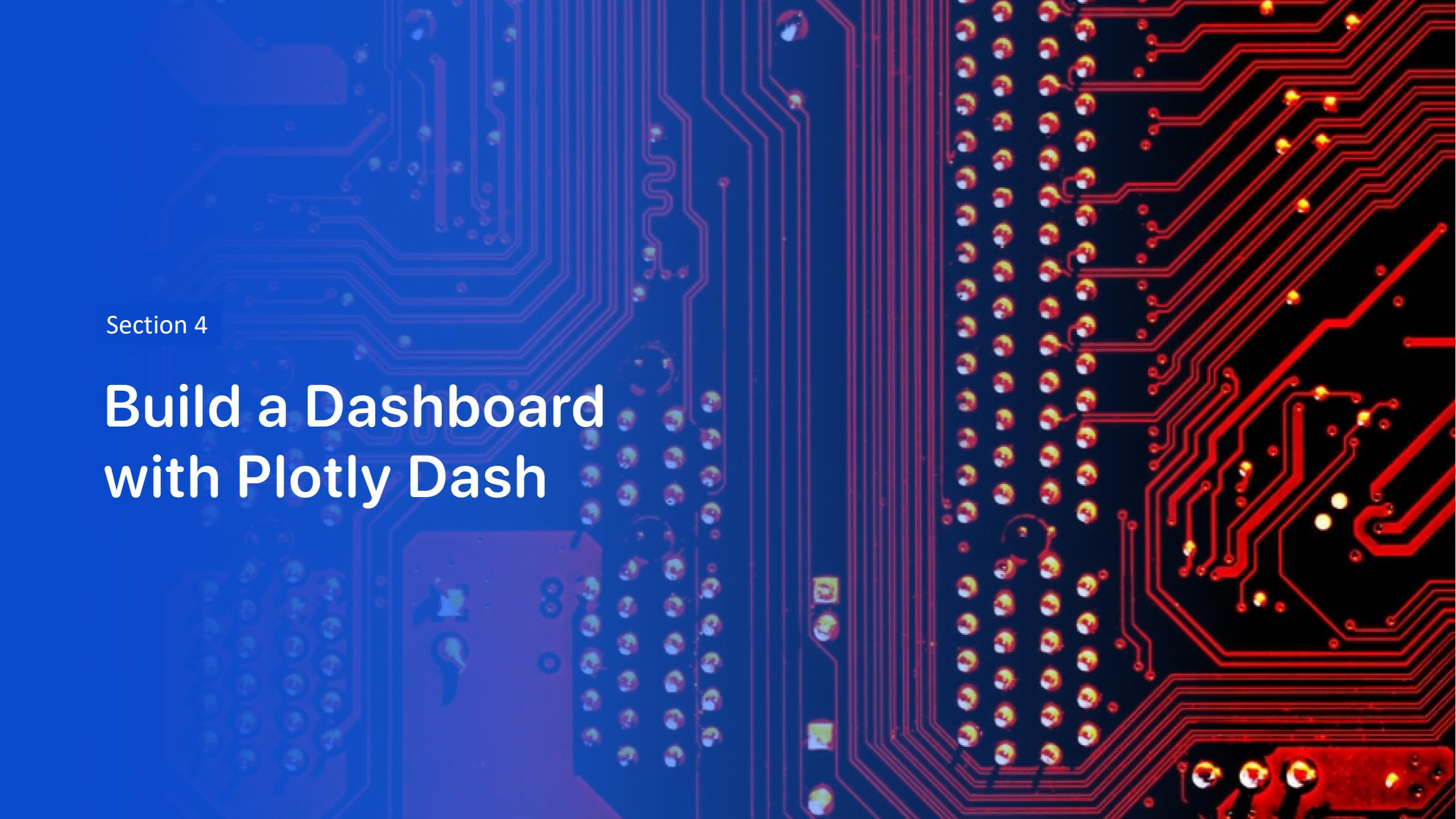
Railways



Coastline



City



Section 4

Build a Dashboard with Plotly Dash

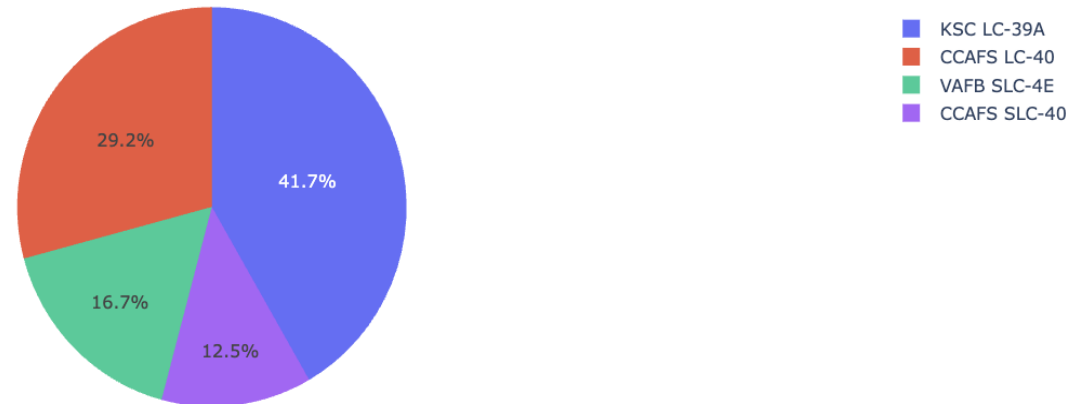
Total Success Launches by All site

SpaceX Launch Records Dashboard

All Sites

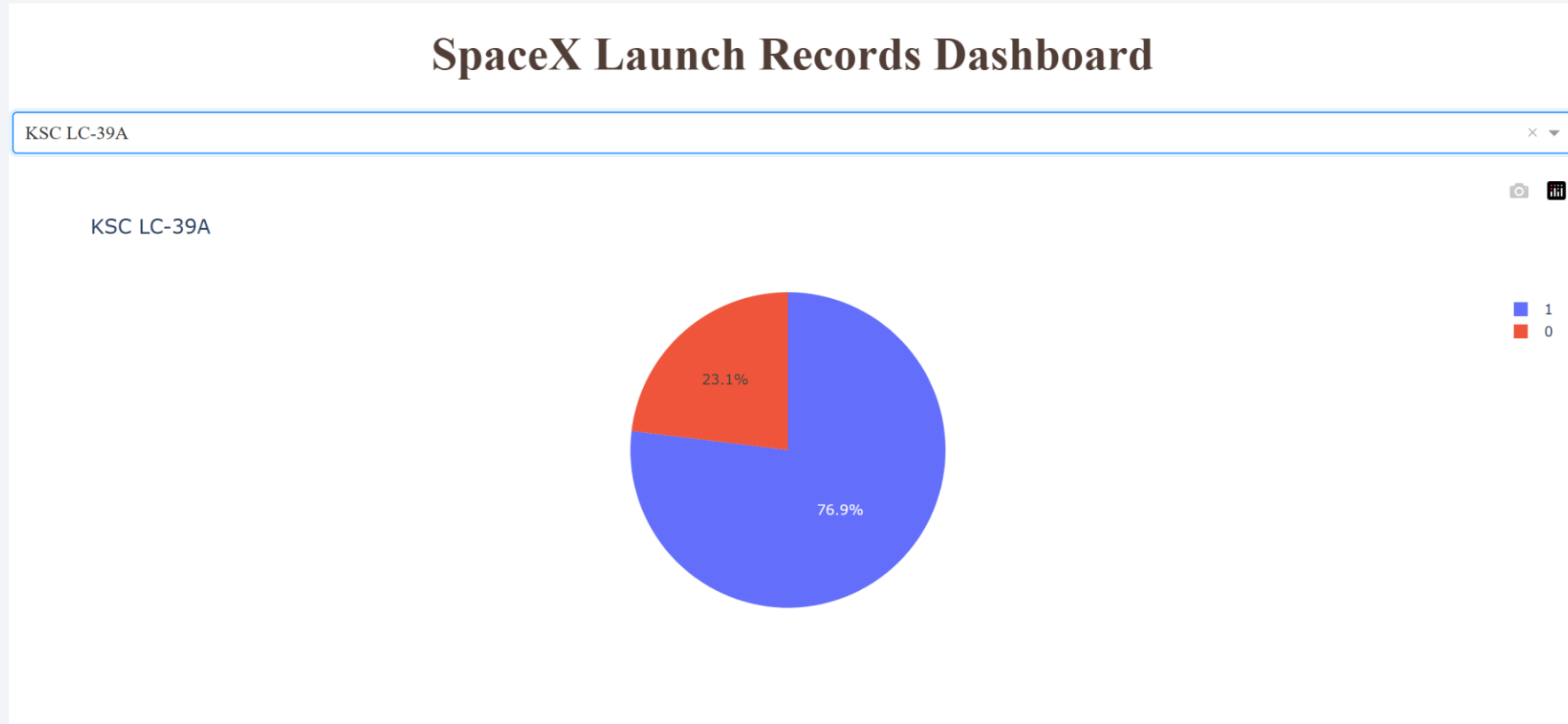


Total Success Launches by Site ALL



KSC LC-39A had the most successful launches from all the sites.

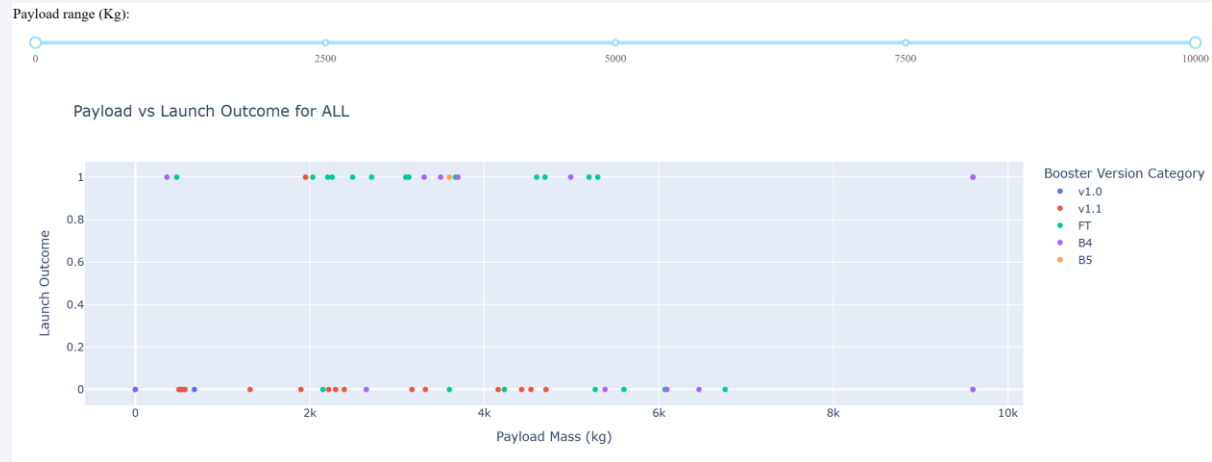
KSC LC-39A is the highest launch site successfully



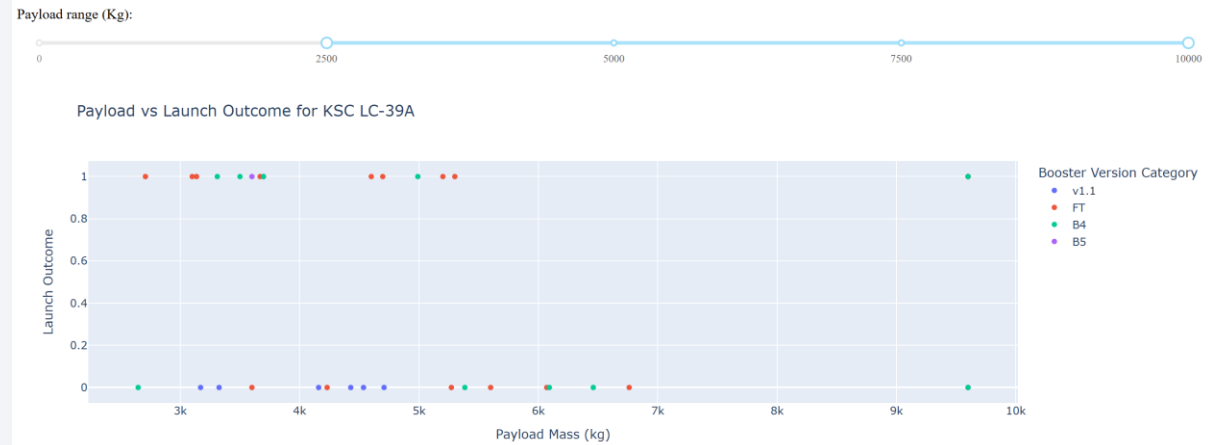
Explain the important elements and findings on the screenshot

Payload vs. Launch Outcome

- The range 2500-5000(KG) concentrate the majority of the successfully launches,
- The range 0-2500(KG) has most failed launches



All site

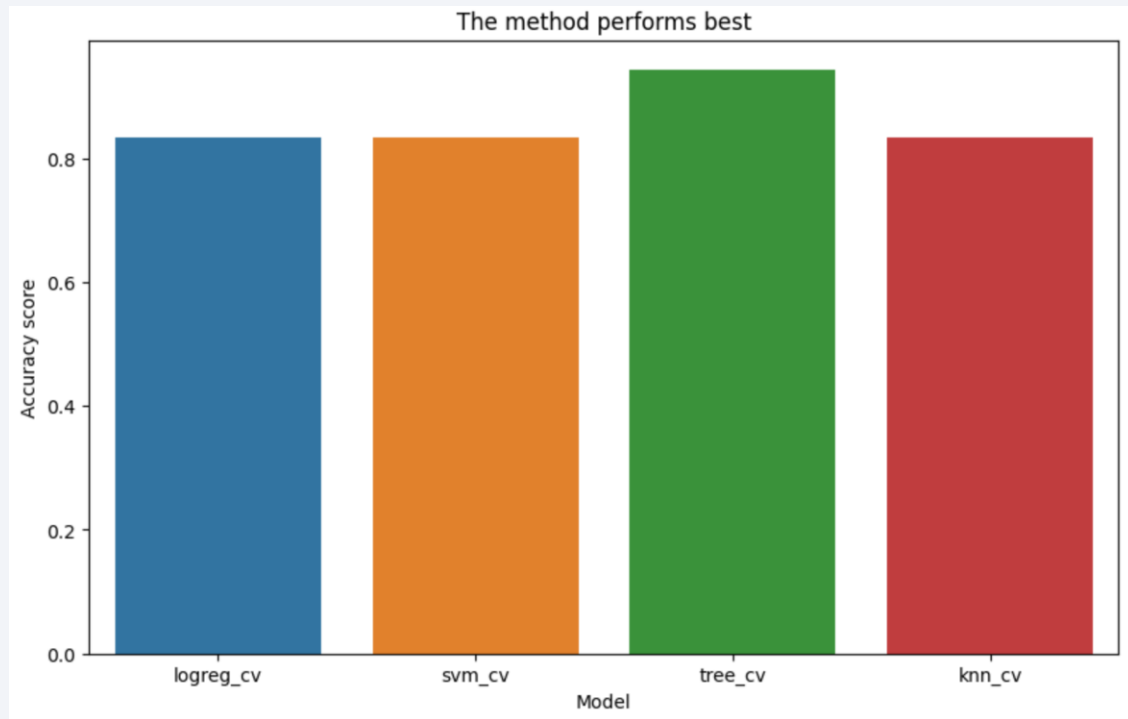


KSC LC-39A

Section 5

Predictive Analysis (Classification)

Classification Accuracy



The decision tree classifier is the best model with highest accuracy

```
import matplotlib.pyplot as plt
import seaborn as sns

df_bar = pd.DataFrame({
    'Model': ['logreg_cv', 'svm_cv', 'tree_cv', 'knn_cv'],
    'Accuracy': [test_score1, test_score2, test_score3, test_score4]
})

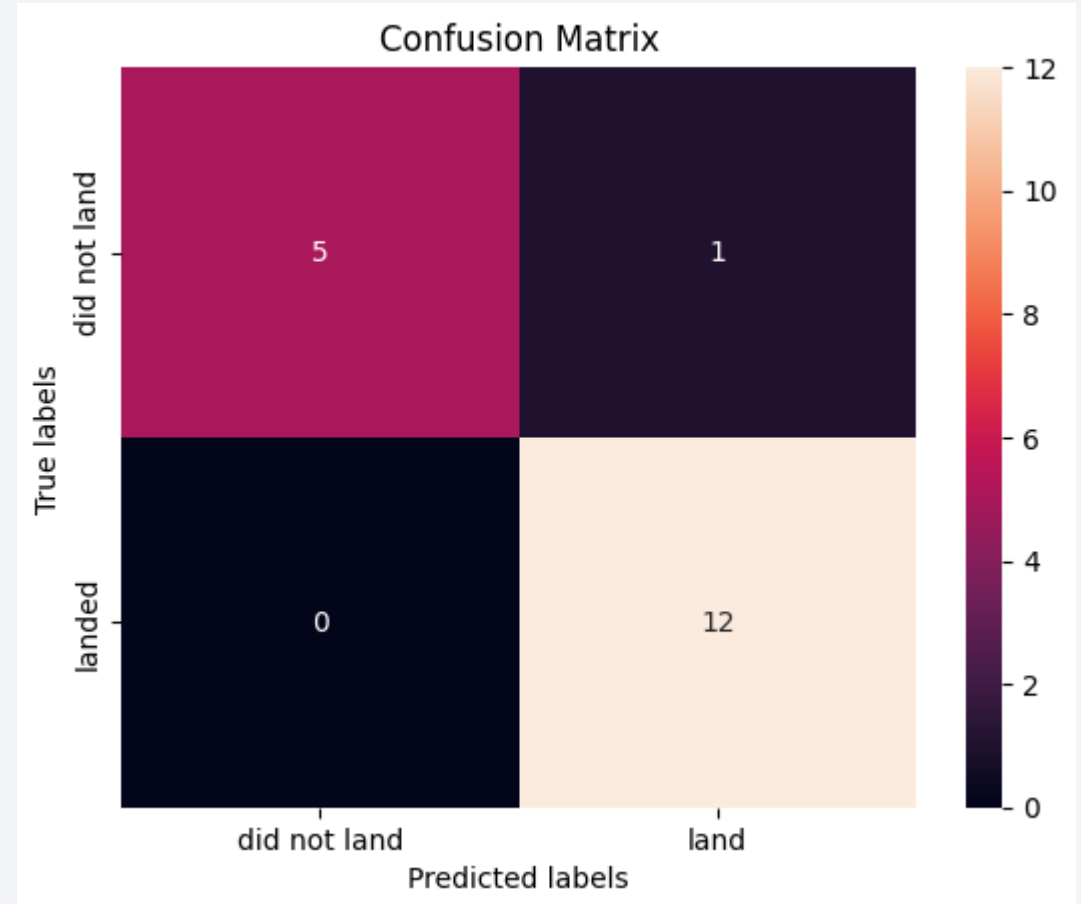
plt.figure(figsize=(10,6))
sns.barplot(x='Model', y='Accuracy', hue='Model', data=df_bar)
plt.title('The method performs best')
plt.xlabel('Model')
plt.ylabel('Accuracy score')
plt.show()
```

Python

Confusion Matrix

Tree Classifier is the best model, as you can see there is only 1 mistake FP when predict on the test set.

- True Positive - 12 (True label is landed, Predicted label is also landed)
- False Positive - 1 (True label is not landed, Predicted label is landed)



Conclusions

- As the number of flights increases, the first stage is more likely to land successfully.
- The launch success rate steadily increased from 2013 to 2020.
- The VELO, ES-L1, GEO, HEO, and SSO orbits show the highest success rates.
- KSC LC-39A has the highest number of successful launches among all launch sites.
- Launch sites are typically located near railways, roads, highways, and coastlines to support logistics.
- Payloads in the 2,500–5,000 kg range account for the majority of successful launches.
- The Decision Tree model achieves the highest accuracy among all classifiers when predicting first-stage landing success using the machine learning pipeline.

Thank you!

